SLGA™ Architecture Map

SLGA™ Architecture Map

---------- Forwarded message ---------

From: Knowenger <knowenger@gmail.com> Date: Fri, Oct 16, 2026, 9:20 AM Subject: SLGA™ Architecture Map To: Miroslav Pis <swissstaffingagency@gmail.com>, <Gutflex@gmail.com>, < originopenfoundation@gmail.com>

SLGA™ Architecture Map Software Lifecycle Governance Architecture —

Software Lifecycle Governance Layer Official Software Lifecycle Governance Reference Map for Software Products, Applications, Components, Services, Platforms, AI Systems, Autonomous Software Agents, Digital Infrastructure, Embedded Software, and Future Software Systems

SLGA™ serves as the official OOF® Software Lifecycle Governance reference

map used to identify, define, classify, navigate, govern, evaluate, and understand governable Software Lifecycle spaces across software products, applications, components, services, platforms, AI systems, autonomous software agents, digital infrastructure, embedded software, and future software systems.

The purpose of SLGA™ is not to prescribe a software development methodology, replace software engineering, perform software creation, execute testing, releases, deployments, changes, or retirement operations, independently grant lifecycle authority, replace specialized technical, security, safety, regulatory, contractual, or operational decision systems, or substitute for general evidence, validation, integrity, identity, accountability, or autonomous-system governance.

The purpose of SLGA™ is to identify, define, separate, and map the governable spaces through which Software Requirements are established, Software Creation is governed, Dependencies are identified and controlled, Configurations and Builds remain traceable, Quality and Security conditions are evaluated, Software Releases become ready and legitimately authorized, Deployments are planned and verified, Software Changes are assessed and routed through applicable lifecycle governance, Software Retirement is legitimately completed, and Software Lifecycle Control continuously preserves state consistency, authorized transitions, bounded exceptions, control continuity, and verifiable control effectiveness.

SLGA™ serves as the official Software Lifecycle Governance reference map

for:

- Software Products

- Software Applications - Software Components - Software Libraries - Software Services - Software Platforms - Cloud Software - Enterprise Software - Open-Source Software - Proprietary Software - AI-Enabled Software - AI Systems - Autonomous Software Agents - Multi-Agent Software Systems - Embedded Software - Industrial Software - Digital Infrastructure - Distributed Software Systems - Human-AI Software Environments - Software Supply Chains - Critical Software Systems - Future Software Ecosystems

Core Architectural Principle

Every Software Lifecycle Governance space answers one primary governance

question.

No Software Lifecycle Governance space duplicates the purpose of another

Software Lifecycle Governance space.

Every Software Lifecycle Governance space has a defined beginning, a

defined operational purpose, and a defined governance boundary.

Every Software Lifecycle Governance space becomes applicable when its own governance conditions arise and the necessary preceding inputs have been sufficiently established.

Software Lifecycle Governance spaces may operate concurrently, recur, or require targeted re-entry.

Software Lifecycle Control operates across all lifecycle spaces rather

than constituting an additional sequential operational stage after Retirement.

Together, these Software Lifecycle Governance spaces form the official

Software Lifecycle Governance reference map of SLGA™. Official Software Lifecycle Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-Cutting Governance Layer: Software Lifecycle Control

1. Software Requirements Standard (SRS)

Governed Space: Software Requirements Governance

Core Question

How can software purpose, requirements, scope, changes, and traceability remain identifiable and governable throughout the software lifecycle? Canonical Definition

Software Requirements Standard (SRS) defines the governance conditions

under which software purpose is established, applicable requirements are specified, software scope is bounded, requirements changes are controlled, and requirements remain traceable across software creation, verification, release, deployment, change, and retirement. Governance Boundary

Begins when a software need, objective, or intended function requires explicit requirements governance.

Ends for an initial requirements baseline when software purpose, requirements, scope, and traceability conditions have been sufficiently established to support Software Creation Governance.

Continues whenever requirements are changed, challenged, superseded, or materially affected by subsequent lifecycle developments.

Software Purpose ≠ Verified Requirements.

Requirements Documented ≠ Requirements Traceable.

• Software Creation Standard (SCS)

Governed Space: Software Creation Governance

Core Question

How can software creation remain identifiable, methodologically controlled, attributable, and traceable across human, automated, and autonomous creation processes? Canonical Definition

Software Creation Standard (SCS) defines the governance conditions under

which software creation methods are selected, creators and creation systems are identified, automated and autonomous creation activities remain controlled, software artifacts are established, and development traceability is preserved. Governance Boundary

Begins when sufficiently established software requirements or an authorized software creation objective enters a creation process.

Ends for a defined creation activity when the resulting software artifacts, creation conditions, creator relationships, and development traceability have been sufficiently established to support subsequent lifecycle governance.

Remains applicable to renewed or modified creation activities.

Software Created ≠ Software Governed.

Automated Creation ≠ Autonomous Authority.

• Software Dependency Standard (SDS)

Governed Space: Software Dependency Governance

Core Question

How can software dependencies remain identifiable, origin-traceable, suitable, change-controlled, and governable throughout the software lifecycle? Canonical Definition

Software Dependency Standard (SDS) defines the governance conditions

under which software components and dependencies are identified, their origins and relevant relationships are traced, their suitability is assessed, their introduction and changes are controlled, and dependency continuity and risk remain governed. Governance Boundary

Begins when an identifiable software object incorporates, proposes to incorporate, or materially relies upon another software component, service, library, platform, or technical dependency.

Ends for an individual dependency governance decision when its identity, origin, suitability, applicable introduction or change conditions, and continuing risk and dependency obligations have been sufficiently established.

Continues while the dependency remains materially relevant to the software

lifecycle.

Component Identified ≠ Dependency Suitable.

Dependency Available ≠ Dependency Continuity Guaranteed.

• Software Configuration Standard (SCFS)

Governed Space: Software Configuration Governance

Core Question

How can software configurations, builds, artifacts, and versions remain identifiable, controlled, reproducible where required, and traceable across lifecycle changes? Canonical Definition

Software Configuration Standard (SCFS) defines the governance conditions

under which software configuration baselines, builds, artifacts, versions, configuration changes, and applicable reproducibility or equivalence requirements remain identifiable, controlled, and traceable. Governance Boundary

Begins when software artifacts, components, or settings require representation as an identifiable configuration, build, version, or baseline.

Ends for a defined configuration governance activity when the relevant baseline, build, artifact identity, version relationships, changes, and applicable reproducibility or equivalence conditions have been sufficiently established.

Continues when configurations or their material dependencies change.

Software Version ≠ Software Build.

Configuration Recorded ≠ Configuration Verified.

• Software Quality Standard (SQS)

Governed Space: Software Quality Governance

Core Question

How can software quality requirements, evaluation criteria, testing coverage, defects, and resulting quality conditions remain governed through identifiable evidence? Canonical Definition

Software Quality Standard (SQS) defines the governance conditions under

which software quality requirements and criteria are established, test coverage is governed, defects are identified and managed, and the quality condition of identifiable software is assessed through applicable evidence. Governance Boundary

Begins when a software object, artifact, build, version, or operational condition requires quality governance against applicable requirements or criteria.

Ends for a defined quality assessment when the relevant quality conditions, test coverage, defects, evidence limitations, and resulting quality classification have been sufficiently established to support applicable lifecycle decisions.

Remains applicable when material software changes invalidate or alter earlier quality conclusions.

Testing Performed ≠ Quality Established.

Quality Assessment ≠ Security Assessment.

• Software Security Standard (SSS)

Governed Space: Software Security Governance

Core Question

How can software security requirements, exposures, testing, vulnerabilities, and remediation remain identifiable, controlled, and verifiable throughout the software lifecycle? Canonical Definition

Software Security Standard (SSS) defines the governance conditions under

which software security requirements are established, relevant exposures are assessed, security testing is controlled, vulnerabilities are identified and managed, and security remediation is governed and verified. Governance Boundary

Begins when software creation, dependencies, configurations, operation, or other lifecycle conditions establish applicable software security requirements or material security exposure.

Ends for an individual security governance case when its applicable requirements, exposure conditions, testing results, vulnerability status, remediation obligations, and verification outcomes have received a sufficiently governed disposition.

Continues while material security conditions or unresolved obligations

remain.

Vulnerability Identified ≠ Vulnerability Remediated.

Security Remediation Applied ≠ Remediation Effectiveness Verified.

• Software Release Standard (SRLS)

Governed Space: Software Release Governance

Core Question

Under what governed conditions can an identifiable software release candidate become ready, legitimately authorized, recorded, and subject to withdrawal where required? Canonical Definition

Software Release Standard (SRLS) defines the governance conditions under

which software release candidates are identified and managed, release readiness is evaluated, competent release authorization is established, release records are preserved, and releases may be withdrawn when applicable conditions require it. Governance Boundary

Begins when an identifiable software artifact, build, or version becomes a proposed release candidate.

Ends for a defined release decision when readiness, authorization, release identity, applicable conditions, and release records have been sufficiently established, or when the candidate is rejected, deferred, or withdrawn.

Continues where an authorized release becomes subject to withdrawal or reassessment.

Release Readiness ≠ Release Authorization.

Release Authorization ≠ Deployment Authorization.

• Software Deployment Standard (SDPS)

Governed Space: Software Deployment Governance

Core Question

How can software deployment remain planned, authorized, operationally identifiable, monitored, and recoverable across actual deployment environments? Canonical Definition

Software Deployment Standard (SDPS) defines the governance conditions

under which software deployment is planned, legitimate deployment authorization is established, production versions and configurations are identified, deployment states are monitored, and rollback or recovery is governed where required. Governance Boundary

Begins when an identifiable software release, version, or configuration is proposed for introduction, replacement, modification, or restoration within a defined deployment environment.

Ends for a deployment activity when its authorization, actual resulting state, applicable monitoring conditions, and required recovery or rollback obligations have received a sufficiently governed disposition.

Continues while deployment state or recovery conditions remain materially

relevant.

Deployment Executed ≠ Deployment Verified.

Authorized Release ≠ Verified Production Deployment.

• Software Change Management Standard (SCMS)

Governed Space: Software Change Management Governance

Core Question

How can software changes remain requested, impact-assessed, coordinated, routed through applicable lifecycle governance, and legitimately closed? Canonical Definition

Software Change Management Standard (SCMS) defines the governance

conditions under which software change requests are identified, their impacts are assessed, change activities are coordinated, materially affected lifecycle governance spaces are re-entered, and change cases are legitimately closed through applicable evidence and verification. Governance Boundary

Begins when a proposed, required, observed, or corrective software change requires explicit governance.

Continues through change request management, impact assessment, coordination, applicable lifecycle re-entry, implementation relationships, verification, and closure.

Ends for an individual change case when its applicable requirements, outcomes, unresolved obligations, and closure conditions have been sufficiently governed.

Change Implemented ≠ Change Verified.

Software Change ≠ Complete Lifecycle Restart by default.

• Software Retirement Standard (SRTS)

Governed Space: Software Retirement Governance

Core Question

How can software be legitimately, safely, and verifiably retired when its operational use, support obligations, dependencies, data, users, authority relationships, and residual responsibilities may continue beyond the decision to discontinue it? Canonical Definition

Software Retirement Standard (SRTS) defines the governance conditions

under which software retirement is planned, end-of-support conditions are established, migration is governed where applicable, residual dependencies and continuing obligations are identified and managed, and retirement closure is verified. Governance Boundary

Begins when an identifiable software object becomes subject to proposed, required, or initiated retirement governance.

Continues through retirement planning, support disposition, migration where applicable, residual dependency management, and closure verification.

Ends active retirement governance for the applicable software scope when a sufficiently supported Software Retirement Closure State has been established and continuing obligations have received an appropriate governance disposition.

Retirement Decision ≠ Retirement Completion.

End of Support ≠ End of Operational Use.

Retirement Closure ≠ Historical Evidence Erasure.

• Software Lifecycle Control Standard (SLCS)

Governed Space: Software Lifecycle Control Governance

Core Question

How can software remain identifiable, controllable, and governable throughout its lifecycle when its states, transitions, actors, dependencies, operational conditions, authority relationships, and governance requirements continuously change? Canonical Definition

Software Lifecycle Control Standard (SLCS) defines the governance

conditions under which software lifecycle states remain consistent, lifecycle transitions occur under legitimate authority and verifiable conditions, lifecycle exceptions remain bounded, control transfers preserve authority and operational continuity, and lifecycle controls are assessed for suitability, implementation, and actual effectiveness. Governance Boundary

Begins when an identifiable software object becomes subject to lifecycle governance and requires consistency between its declared state, actual condition, applicable requirements, authority, and control arrangements.

Operates continuously across Software Requirements, Creation, Dependencies,

Configuration, Quality, Security, Release, Deployment, Change Management, and Retirement.

Continues through lifecycle state changes, authorized transitions, exceptions, control transfers, control assessments, and applicable reassessment.

Ends for a defined control case when its state, control requirements, verification outcomes, and continuing obligations have received an appropriate governance disposition.

SLCS is cross-cutting and is not a sequential operational phase after Software Retirement.

Declared Lifecycle State ≠ Verified Lifecycle State.

Technical Transition Capability ≠ Legitimate Transition Authority.

Control Implemented ≠ Control Effective. Core Governed Software Lifecycle Spaces

- Software Requirements

- Software Creation - Software Dependencies - Software Configuration - Software Quality - Software Security - Software Release - Software Deployment - Software Change Management - Software Retirement - Software Lifecycle Control

Complete Software Lifecycle

SLGA™ does not treat Software Creation, Release, or Deployment as completion of the software lifecycle.

The complete governed lifecycle is:

Software Purpose → Requirements → Creation → Dependencies → Configuration

→ Quality → Security → Release → Deployment → Change Management → Retirement

Software Lifecycle Control operates across the complete lifecycle.

Software may therefore remain subject to governance after its initial release and deployment.

A release establishes a governed software release state.

It does not establish that deployment has occurred.

A deployment establishes an operational software state.

It does not establish that the complete software lifecycle has ended.

A retirement decision establishes an intended lifecycle disposition.

It does not establish that retirement has been completed.

Software Creation ≠ Software Lifecycle Completion.

Software Release ≠ Software Lifecycle Completion.

Software Deployment ≠ Software Lifecycle Completion.

Retirement Decision ≠ Retirement Completion. Recursive Software Lifecycle

SLGA™ does not assume that governed software progresses through its lifecycle only once.

Once software has been created, its lifecycle conditions may continue to change:

Software Requirements may change.

Software Artifacts may change.

Dependencies may change.

Configurations may change.

Quality Conditions may change.

Security Conditions may change.

Release Authorization may change.

Deployment States may change.

Operational Environments may change.

Responsible Actors may change.

Control Authority may change.

Lifecycle Exceptions may arise or expire.

Previously effective controls may become ineffective.

Retirement conditions may change.

Residual dependencies may require continuing governance.

The architecture therefore becomes recursive:

Governed Software → Software Change → Change Impact Assessment →

Applicable Lifecycle Re-Entry → Renewed Governance → Release / Deployment where applicable → Continued Lifecycle Governance

Where software enters retirement:

Governed Software → Retirement Planning → End-of-Support Management →

Migration where applicable → Residual Dependency Management → Retirement Closure

Where materially relevant software reality changes:

Existing Lifecycle State → Material Change → Earliest Materially Affected

SLGA™ Governance Space → Reassessment → Updated Lifecycle State

A materially changed software purpose or requirement may require re-entry into Software Requirements Standard.

A new creator, changed creation method, or materially altered software

creation process may require re-entry into Software Creation Standard.

A new, changed, compromised, unavailable, or incorrectly identified

dependency may require re-entry into Software Dependency Standard.

A changed build, version, artifact, or configuration baseline may require re-entry into Software Configuration Standard.

New defects, changed quality criteria, or invalidated testing evidence may require re-entry into Software Quality Standard.

New vulnerabilities, changed exposure conditions, or incomplete remediation

may require re-entry into Software Security Standard.

A changed release candidate, invalidated readiness condition, revoked

authorization, or release withdrawal may require re-entry into Software Release Standard.

A changed production version, deployment environment, authorization,

rollback, or recovery condition may require re-entry into Software Deployment Standard.

A new or materially altered software change request, impact, coordination

requirement, or closure condition may require renewed governance within Software Change Management Standard.

Changed support conditions, migration requirements, residual dependencies,

or retirement closure conditions may require re-entry into Software Retirement Standard.

A conflicting lifecycle state, unauthorized transition, changed exception,

failed control transfer, or ineffective lifecycle control may require renewed governance within Software Lifecycle Control Standard.

The architecture therefore supports targeted governed Software Lifecycle

Re-Entry rather than uncontrolled reconstruction of the entire software lifecycle.

Re-enter the earliest materially affected governance space.

Reassess what materially changed. Preserve what remains valid.

Lifecycle re-entry must preserve historical traceability. Software Identity & Configuration Structure

SLGA™ does not assume that one software product represents one stable artifact, configuration, version, or operational state.

A single software product may contain:

multiple Software Components

multiple Software Dependencies

multiple Software Artifacts

multiple Builds

multiple Versions

multiple Configurations

multiple Release Candidates

multiple Authorized Releases

multiple Deployment Instances

multiple Operational Environments

multiple simultaneous Lifecycle States

or:

a wider distributed Software System

Each software object may possess its own lifecycle attributes while remaining connected to the broader software structure.

One release may be deployed across multiple environments.

Different environments may contain different versions.

A component may be retired in one environment while remaining operational in another.

A software service may remain active while one of its dependencies becomes unsupported.

Therefore:

One Software Product ≠ One Software Version.

One Software Version ≠ One Deployment Instance.

Release State ≠ Deployment State.

Declared State ≠ Verified State.

Local Software Retirement ≠ Complete System Retirement. Architectural Position

SLGA™ governs the complete Software Lifecycle Governance lifecycle from

software purpose and requirements formation through legitimate retirement, with continuous control over lifecycle states, transitions, exceptions, authority relationships, and control effectiveness.

The architecture progresses through:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-Cutting Governance Layer: Software Lifecycle Control

Together, these Software Lifecycle Governance spaces govern why software is required, what requirements and scope apply, how it is created, what dependencies it uses, how its configurations and builds remain controlled, what quality and security conditions are established, when release is legitimate, how deployment is authorized and verified, how changes return to applicable governance, how retirement is completed, and whether lifecycle control remains effective throughout the process.

SLGA™ therefore provides a complete governance path from the first

identification of why software should exist and what it must achieve to the establishment of when its operational lifecycle can legitimately conclude while preserving applicable continuing obligations and historical traceability. What SLGA™ Defines

SLGA™ defines:

• where Software Lifecycle Governance begins,

• how Software Purpose is established,

• how Software Requirements are specified,

• how Software Scope is governed,

• how Requirements Changes are controlled,

• how Requirements Traceability is preserved,

• how Development Methods are selected,

• how Software Creators are identified,

• how Automated Software Creation is controlled,

• how Software Artifacts are established,

• how Development Traceability is maintained,

• how Software Components are identified,

• how Dependency Origins are traced,

• how Dependency Suitability is assessed,

• how Component Introduction and Changes are controlled,

• how Dependency Continuity and Risk are governed,

• how Configuration Baselines are established,

• how Software Builds are governed,

• how Artifacts and Versions are identified,

• how Configuration Changes are controlled,

• how Build Reproducibility and Equivalence are assessed,

• how Software Quality Requirements are established,

• how Quality Criteria are defined,

• how Test Coverage is governed,

• how Software Defects are managed,

• how Software Quality Conditions are assessed,

• how Software Security Requirements are established,

• how Software Exposure is assessed,

• how Security Testing is controlled,

• how Vulnerabilities are managed,

• how Security Remediation is governed,

• how Release Candidates are managed,

• how Release Readiness is established,

• how Release Authorization is governed,

• how Release Records are maintained,

• how Release Withdrawal is governed,

• how Deployment Planning is established,

• how Deployment Authorization is governed,

• how Production Versions are controlled,

• how Deployment States are monitored,

• how Rollback and Recovery are governed,

• how Change Requests are managed,

• how Change Impacts are assessed,

• how Change Coordination is controlled,

• how Lifecycle Re-Entry is governed,

• how Change Closure is established,

• how Software Retirement is planned,

• how End-of-Support conditions are managed,

• how Software Migration is governed,

• how Residual Dependencies are managed,

• how Retirement Closure is verified,

• how Lifecycle State Consistency is controlled,

• how Lifecycle Transitions are governed,

• how Lifecycle Exceptions are managed,

• how Lifecycle Control Transfers preserve continuity,

• how Lifecycle Control Effectiveness is assessed,

• and how identifiable software remains governed throughout its complete lifecycle.

SLGA™ provides the official Software Lifecycle Governance language used for

software lifecycle identification, navigation, governance classification, structural diagnosis, lifecycle state analysis, change governance, retirement governance, control assessment, and architecture design across human-developed, automated, AI-enabled, autonomous, embedded, industrial, enterprise, distributed, infrastructural, and future software environments. Critical Architectural Distinctions

Software Lifecycle Governance ≠ Software Engineering Execution

Software Created ≠ Software Governed

Software Purpose ≠ Verified Requirements

Requirements Documented ≠ Requirements Traceable

Software Generated ≠ Creation Authority Established

Automated Creation ≠ Autonomous Authority

Creator Identified ≠ Creation Authorized

Component Identified ≠ Dependency Suitable

Dependency Origin Known ≠ Dependency Risk Resolved

Dependency Available ≠ Dependency Continuity Guaranteed

Software Product ≠ Software Component

Software Version ≠ Software Build

Software Build ≠ Deployment Instance

Configuration Recorded ≠ Configuration Verified

Build Reproducibility ≠ Software Quality

Testing Performed ≠ Quality Established

Quality Assessment ≠ Security Assessment

Vulnerability Identified ≠ Vulnerability Remediated

Security Remediation Applied ≠ Remediation Effectiveness Verified

Release Candidate ≠ Authorized Release

Release Readiness ≠ Release Authorization

Release Authorization ≠ Deployment Authorization

Release Authorized ≠ Actual Deployment Verified

Deployment Executed ≠ Deployment Verified

Deployment Successful ≠ Lifecycle Control Effective

Change Requested ≠ Change Authorized

Change Implemented ≠ Change Verified

Change Verified ≠ Change Closed by default

Software Change ≠ Complete Lifecycle Restart by default

Lifecycle Re-Entry ≠ New Software Identity by default

Retirement Decision ≠ Retirement Completion

End of Support ≠ End of Operational Use

Software Migration ≠ Residual Dependency Elimination

Operational Retirement ≠ Elimination of Continuing Governance Obligations

Declared Lifecycle State ≠ Verified Lifecycle State

Technical Transition Capability ≠ Legitimate Transition Authority

Transition Executed ≠ Transition Verified

Exception Requested ≠ Exception Approved

Exception Authorized ≠ Requirement Eliminated

Exception Approved ≠ Residual Risk Resolved

Responsibility Assigned ≠ Authority Established

Authority Established ≠ Capability Available

Control Transferred Administratively ≠ Control Continuity Verified

Control Documented ≠ Control Implemented

Control Implemented ≠ Control Effective

Control Activity Recorded ≠ Control Objective Achieved

Agent Instruction ≠ Enforced Authority Boundary

Task Authority ≠ Constituent Action Authority

Current Software State ≠ Complete Software Lifecycle History

Cross-Architecture Governance Boundaries SLGA™ ↔ GOA™ — Governance Architecture

GOA™ — Governance Architecture governs the constitutional structure,

authority, boundaries, delegation, validation, and continuity of governance.

SLGA™ — Software Lifecycle Governance Architecture governs the software

lifecycle under applicable governance requirements and legitimate authority.

General Governance Authority ≠ Software Lifecycle Authorization. SLGA™ ↔ ORA™ — Operational Reality Architecture

ORA™ — Operational Reality Architecture governs what exists and occurs in

actual Operational Reality.

SLGA™ — Software Lifecycle Governance Architecture governs the software

lifecycle conditions, states, decisions, and controls applicable to that operational reality.

Declared Software State ≠ Actual Operational State. SLGA™ ↔ AGA™ — Accountability Governance Architecture

AGA™ — Accountability Governance Architecture governs responsibility,

accountability, attribution, authority, and related accountability relationships.

SLGA™ — Software Lifecycle Governance Architecture governs software

lifecycle requirements and controls involving those relationships.

Responsibility Assigned ≠ Lifecycle Control Effective. SLGA™ ↔ ADIT® — Continuous Audit & Evidence Governance Architecture

ADIT® governs evidential observation, preservation, correlation,

verification, examination, and auditability.

SLGA™ governs software lifecycle requirements, decisions, states, and

controls supported by relevant evidence.

Evidence Recorded ≠ Lifecycle Requirement Satisfied. SLGA™ ↔ VALIDOS™ — Validation Governance Architecture

VALIDOS™ — Validation Governance Architecture governs whether a

validation basis sufficiently supports a Validation Determination.

SLGA™ — Software Lifecycle Governance Architecture governs software

lifecycle decisions and requirements that may depend on validated information.

Validation Determination ≠ Release Authorization. SLGA™ ↔ INTEGROS® — Integrity Governance Architecture

INTEGROS® — Integrity Governance Architecture governs integrity across

objects, states, evidence, and processes.

SLGA™ — Software Lifecycle Governance Architecture governs software

lifecycle requirements and controls.

General Integrity Governance ≠ Software Lifecycle Governance. SLGA™ ↔ OBIDENITY® — Origin-Bound Identity Governance Architecture

OBIDENITY® — Origin-Bound Identity Governance Architecture governs

Origin, Identity, Ownership, Authority, Permission, Responsibility, Provenance, Transfer, Continuity, and Auditability.

SLGA™ — Software Lifecycle Governance Architecture governs identifiable

software objects and their lifecycle relationships.

Software Identity ≠ Software Lifecycle State. SLGA™ ↔ AIG® — AI Governance Architecture

AIG® — AI Governance Architecture governs AI behavioral governance and

behavioral integrity.

SLGA™ — Software Lifecycle Governance Architecture governs the software

lifecycle of AI-enabled software.

AI Behavior ≠ Software Lifecycle Governance. SLGA™ ↔ ASGA™ — Autonomous Systems Governance Architecture

ASGA™ — Autonomous Systems Governance Architecture governs

autonomous-system authority, execution, constraints, coordination, intervention, escalation, and operational governability.

SLGA™ — Software Lifecycle Governance Architecture governs software

lifecycle requirements and controls applicable to autonomous software systems.

Autonomous Execution Capability ≠ Software Lifecycle Authority.

Autonomous Action ≠ Authorized Lifecycle Transition. SLGA™ ↔ SIMULOS® — Simulation Governance Architecture

SIMULOS® — Simulation Governance Architecture governs simulation purpose,

structure, execution, outputs, and lifecycle.

SLGA™ — Software Lifecycle Governance Architecture governs actual

software lifecycle objects, decisions, and states.

Simulated Deployment ≠ Actual Deployment.

Simulated Control Success ≠ Verified Operational Control Effectiveness. SLGA™ ↔ Software Engineering Systems

SLGA™ — Software Lifecycle Governance Architecture governs the software

lifecycle.

Software engineering systems govern coding, technical implementation,

testing execution, build operations, deployment execution, maintenance activities, and related engineering workflows.

Software Engineering Execution ≠ Software Lifecycle Governance. SLGA™ ↔ Software Security Systems

SLGA™ — Software Lifecycle Governance Architecture governs the software

lifecycle, including software security requirements, exposures, testing control, vulnerability management, and remediation governance.

Specialized software security systems perform technical detection,

analysis, protection, testing, and remediation activities.

Security Tool Execution ≠ Security Governance Completion. Compatible OOF® Architectures

- GOA™ — Governance Architecture

- ORA™ — Operational Reality Architecture - AGA™ — Accountability Governance Architecture - ADIT® — Continuous Audit & Evidence Governance Architecture - VALIDOS™ — Validation Governance Architecture - INTEGROS® — Integrity Governance Architecture - OBIDENITY® — Origin-Bound Identity Governance Architecture - AIG® — AI Governance Architecture - ASGA™ — Autonomous Systems Governance Architecture - SIMULOS® — Simulation Governance Architecture

Canonical Principle

SLGA™ does not prescribe software development methodologies, replace software engineering, perform software creation, execute testing, release, deployment, change, or retirement operations, independently grant lifecycle authority, replace specialized security or quality requirements, govern general accountability, perform general evidence governance, or substitute for contractual, regulatory, safety, technical, or other domain-specific decision systems. SLGA™ governs the methodological conditions under which identifiable software becomes requirements-grounded, creation-traceable, dependency-aware, configuration-controlled, quality-assessed, security-governed, release-authorized, deployment-verifiable, change-controlled, legitimately retired, and continuously governed through consistent lifecycle states, authorized transitions, bounded exceptions, preserved control continuity, and evidence-supported control effectiveness. Canonical Closing Statement

SLGA™ serves as the official Software Lifecycle Governance reference map

for software products, applications, components, services, platforms, AI systems, autonomous software agents, digital infrastructure, embedded software, and future software systems. By defining the primary governable spaces of the complete Software Lifecycle, SLGA™ provides the architecture used to establish software purpose and requirements, govern creation, identify and control dependencies, preserve configuration and version traceability, evaluate quality and security conditions, establish legitimate release authorization, govern deployment and actual operational state, manage software changes and targeted lifecycle re-entry, establish retirement conditions and closure, and preserve lifecycle state consistency, legitimate transitions, bounded exceptions, control continuity, and verifiable control effectiveness across the complete software lifecycle. SLGA™ therefore establishes Software Lifecycle Governance as a distinct methodological layer around software itself, enabling software to remain identifiable, requirements-grounded, creation-traceable, dependency-aware, configuration-controlled, quality-assessed, security-governed, release-authorized, deployment-verifiable, change-controlled, legitimately retired, and continuously governable regardless of whether software is developed by humans, generated through automation, operated through AI systems, modified by autonomous agents, deployed across distributed infrastructure, or maintained within future software ecosystems.

SLGA™ governs Software from Requirements to Retirement — with continuous

control across the entire lifecycle.